

Ejercicio 1

Inciso (a)

Si para todas las observaciones del dataset esa neurona recibe valores de preactivación $z = \mathbf{w}^\top \mathbf{x} + \mathbf{b} \leq 0$, cuando se active seguirá siendo cero el output por estar usando ReLU. Esto se ve únicamente viendo la salida de esa neurona.

Inciso (b)

Una neurona puede quedar muerta si sus pesos y biases hacen que la preactivación $z \leq 0$ para la mayoría o todas las observaciones del dataset. Puede suceder cuando se inician mal, el learning rate es muy alto (hace que los pesos sean valores negativos de golpe) o que la neurona no pueda detectar algunos valores de las observaciones.

Inciso (c)

1. Cambiar ReLU y usar en su lugar otra función de activación que no permita que se active la neurona con valores exactamente cero. Esto podría generar el problema de vanishing gradient de todas formas. Si queremos mantener la facilidad de entrenamiento, usaría Leaky ReLU, ELU o Swish.
2. Normalizar o estandarizar los datos o features para evitar que todas las preactivaciones sean menores o iguales a cero.
3. Inicialización de pesos adecuada.
4. Definir un learning rate adecuado antes del entrenamiento.

Inciso (d)

Si se inician pesos con valores negativos grandes, puede suceder que la activación de z sea negativa o cero y muera. El hiperparámetro del learning rate seteado a un valor moderado disminuye la probabilidad de que muera la neurona.

Ejercicio 3

Inciso (a)

La función **sigmoide** sufre el problema de vanishing gradient, causada por la saturación en los extremos. La saturación se da cuando la función se acerca cada vez más al cero.

Inciso (b)

La función **tanh** también sufre del problema de vanishing gradient por la saturación.

Inciso (c)

La función **softmax** es una función de activación que se hace únicamente con las neuronas de output. Se usa para normalizar los logits (valores de salida antes de aplicar softmax) para conseguir una distribución de probabilidad. Los logits pueden ser positivos, negativos, grandes o pequeños. Luego de aplicar la función softmax, la suma de los logits activados suman 1.

Inciso (d)

El rango de valores que produce la función es $0 < \text{softmax}(x_i) < 1$.

Inciso (e)

Calcular directamente la función softmax puede causar inestabilidad numérica por tener logits muy grandes y puede causar overflow. Para evitarlo se usa **log-softmax**, que usa un logaritmo para generar esa estabilidad que buscamos.

Ejercicio 5

- **Falso.** Es diferenciable en todo su dominio, menos en $x = 0$.
- **Verdadero.** Puede producir neuronas que queden permanentemente inactivas porque devuelven cero siempre. Ocurre cuando la preactivación es siempre negativa y la neurona deja de recibir gradiente.
- **Falso.** Se usa únicamente en la capa de salida para producir probabilidades cuya suma conjunta es igual a uno.
- **Falso.** Se usan en cada neurona de las capas ocultas y de la capa de salida.
- **Verdadero.** Está diseñada para que no tenga nunca gradiente cero.
- **Falso.** Una función lineal no restringe la salida entre 0 y 1. Esta función no produce probabilidades y no permite usar funciones de pérdida adecuadas. Para clasificación conviene usar la función de activación sigmoide σ .
- **Verdadero.** En caso de no tener funciones de activación, una red neuronal colapsaría (sería solamente una combinación de transformaciones lineales) y se necesitaría una sola capa. Además, no aprendería relaciones no lineales. No es más que una regresión lineal.
- **Falso.** En la práctica no se usa la función sigmoide más que nada por los problemas que atrae: vanishing gradient y el hecho de que no esté centrada en cero (todos los outputs son positivos).
- **Falso.** Tiene un rango de salida entre -1 y 1.